# Vehicle Visual Defect Detection

By

Sirinda Leesuravanich

Kaiqing Qi

Reethesh Venkataraman

David Wei

Supervisor: Igor Yakushin, PhD

A Capstone Project

Submitted to the University of Chicago in partial fulfillment

of the requirements for the degree of

Master of Science in Applied Data Science

Division of Physical Sciences

December 2025

# Abstract

This project develops an end-to-end automated vehicle defect detection system using the YOLO11-XL model with oriented bounding boxes (OBB). Trained on the CarDD dataset, the system incorporates data preprocessing, label correction, OBB conversion, and targeted augmentation to improve detection of subtle defects. The final model achieved strong performance across all six defect categories, with the lowest class exceeding 0.65 mAP@0.5. The application framework integrates SSIM-based frame extraction to convert video into model-ready frames and visualize outputs through a web interface. The results demonstrate the feasibility of deploying computer vision model for real-world automotive quality control.



*Keywords*: computer vision, YOLOv11, vehicle defect detection, Oriented Bounding Box (OBB), manufacturing quality control, CarDD dataset

# Executive summary

This project presents an end-to-end AI system for automated vehicle defect detection designed to improve accuracy and consistency in automotive inspection workflows, reducing reliance on manual visual checks that are prone to human error and inconsistency. Using the Car Damage Detection (CarDD) dataset, we built a complete data pipeline and trained a YOLO11-XL model with oriented bounding boxes (OBB) to detect six major defect types. Through data preprocessing, targeted augmentation, and model engineering, the system achieved strong performance across all classes, surpassing 0.65 mAP@0.5.

In addition, the project includes an application prototype that integrates SSIM-based frame extraction with a visualization interface to support efficient review of model predictions and real inspection workflows.

Overall, the project demonstrates the feasibility of deploying OBB-based computer vision for automotive quality control. With domain-specific data and cleaner annotations, the system can be further improved and scaled. The final handoff positions LatentView Analytics to refine and commercialize this solution for automotive, insurance, and fleet operations.

# Table of Contents

**ABSTRACT** ..... **I**

**EXECUTIVE SUMMARY** ..... **II**

**TABLE OF CONTENTS** ..... **III**

**INTRODUCTION** ..... **1**

PROBLEM STATEMENT ..... 1

ANALYSIS GOALS ..... 1

SCOPE ..... 2

**BACKGROUND** ..... **3**

LITERATURE REVIEW SUMMARY ..... 5

**DATA** ..... **6**

DATA SOURCES ..... 6

DESCRIPTIVE ANALYSIS ..... 7

**METHODOLOGY** ..... **10**

FEATURE ENGINEERING ..... 10

MODELLING FRAMEWORK ..... 15

*APPLICATION ARCHITECTURE AND SYSTEM WORKFLOW* ..... 18

*Frame Extraction Using Structural Similarity Index (SSIM)* ..... 22

**FINDINGS** ..... **23**

**DISCUSSION** ..... **25**

**CONCLUSION** ..... **28**

**REFERENCES** ..... **33**

# Introduction

## Problem Statement

Vehicle quality inspection is a significant part of vehicle manufacturing operations today. It aims to catch all visual defects on newly manufactured cars before selling to customers, since a dent in a newly bought car will provoke customer dissatisfaction, product returns, and negatively impact business performance. Currently, the traditional vehicle inspection process is heavily reliant on manual tasks that are prone to human error and subjectivity, as they depend on the individual judgment, experience, and carefulness of inspectors. Even the best inspector would grow tired and might let defects slip through.

The objective of this project is to develop an AI-powered defect detection system that leverages computer vision and automation to augment the manual inspection process. This system will analyze images captured from real-time video feeds along the manufacturing line to identify visual defects in the final stage of production. By improving inspection accuracy, consistency, and speed, the solution aims to reduce operational costs and enhance product quality.

## Analysis Goals

The primary goal of this analysis is to develop a scalable, end-to-end computer vision solution capable of accurately detecting, classifying, and locating visual defects on vehicles using video data captured from manufacturing-line cameras. The system is intended to enhance

the precision, efficiency, and traceability of visual quality inspections by reducing reliance on manual reviews, which are often inconsistent and labor-intensive.

To achieve this objective, the proposed solution must be able to process video inputs, extract meaningful frames for analysis, detect and classify relevant defects, and present the results in a format that supports rapid inspection and decision-making. The inclusion of a web-based interface further enables inspectors to review detections, navigate frames efficiently, and manage inspection outputs in a structured and user-friendly manner.

In the initial phase of this project, the goal is to develop a prototype model using publicly available datasets of damaged vehicle images. This proof-of-concept demonstrates the core capabilities of automated defect detection and provides the client with a tangible artifact to showcase to customers. It serves as an early validation of the feasibility and potential impact of an AI-powered inspection system within manufacturing and quality-control workflows.

**Scope**

Due to limited resources on Colab Pro, particularly memory constraints, we had to scope the model tuning parameters, especially the input image size. This limitation affects the overall accuracy of the model. Therefore, we optimized the model to ensure it can be trained and executed efficiently within the available Colab Pro environment.

Additionally, because of the limited availability of publicly accessible datasets, our model was trained on data that is not entirely representative of the common defects found in real-world manufacturing inspections. To tailor our model to be better suited for our use case, we

focused on damages that match more closely: namely scratches, dents, and cracks. However, they are still not perfect representation of what the model might encounter in the manufacturing industry, so we should be careful directly deploying the model (We might suggest fine-tuning our trained weights with direct data from manufacturing companies).

Lastly, the current scope of the project has been limited due to time and resource constraints from the client. Although the initial plan was to deploy the Django-based video processing web application on Google Cloud Platform (GCP), this will not be feasible within the remaining timeline. As a result, the application will be hosted locally as a prototype version. Despite being locally hosted, the prototype will retain full functionality, including video upload, frame extraction, and an interactive interface that allows users to view and navigate through video frames. This approach ensures that all core features are demonstrated effectively, even without cloud deployment.

## Background

LatentView Analytics, established in 2006, is a leading digital analytics firm renowned for its expertise in integrating traditional and digital data to enhance decision-making across industries. With headquarters in India and extensive operations spanning the U.S., Europe, and Asia, LatentView provides advanced analytics solutions to Fortune 500 companies, optimizing their business processes through innovative and data-driven methodologies. Its strong track record has earned recognition in prestigious lists like the Deloitte Technology Fast 50 India and

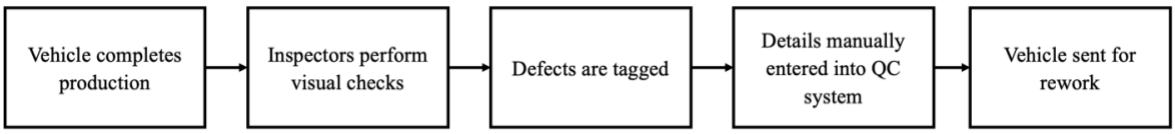
Fast 500 Asia Pacific. With a focus on innovation and client success, LatentView has become a trusted name in analytics, helping organizations unlock the full potential of their data.

**ConnectedView** is LatentView's supply chain transformation philosophy and proprietary analytics framework that delivers a holistic, cross-functional view of business performance by integrating data across functions, channels, and systems. By empowering supply chain professionals with faster, data-driven decision-making, ConnectedView enables more agile and effective operational strategies. The framework brings together accelerators, real-time data integrations, AI/ML models, and analytics delivery capabilities through three key components:

1. **Connected Optimization**: Streamlines operations by optimizing end-to-end business processes such as supply chain, inventory management, and resource allocation.
2. **Connected Planning**: Helps organizations anticipate demand, align resources, and plan proactively across departments.
3. **Connected Visibility**: Enables transparency and real-time insights across the organization by consolidating data from diverse sources.

Background of the traditional vehicle inspection process

**Figure 1.** *Traditional vehicle inspection workflow*




While specific inspection procedures may vary by manufacturer, the traditional workflow in automotive production typically follows a structured, multi-stage process. After a vehicle

completes production on the assembly line, it undergoes a series of manual inspections. An initial inspector visually checks specific sections of the vehicle for defects, followed by one or more additional inspectors who examine other components to ensure quality standards are met. These evaluations are commonly guided by a standardized checklist and rely heavily on the experience of the inspectors. If a defect is identified, it is physically tagged and manually recorded in the quality control system. The vehicle is then routed to a rework station, where necessary corrections are made before it advances to the next stage in the production process.

**Literature Review Summary**

There are several recent research out there focused on detecting car damages by applying computer vison-based models on images of damaged vehicles.

One of the recent papers introduces a dataset called TQVDC, using Three Quarter View Images of damaged cars to do a binary classification of whether this car is damaged. The method they approached the problem with is applying transfer learning to 5 different image classification models and combining their output using a soft voting method, achieving an impressive mAP of > 95% (Lee et al., 2024). However, their problem differs from our goal of analysis. Instead of a binary classification, we also want to detect the location of the visual defect along with a multiclass classification of the type of defect.

One older paper by Dwivedi et al. did attempt to localize the defect in addition to classifying it. Their first approach was to use Class Activation Mapping, which visualizes the pixels of an image that is relatively more responsible for the image classification. They also

attempted to use transfer learning on the YOLO (you-only-look-once) v3 model to localize the defect and identify a bounding box. However, the author states due to their limited data availability, they were struggling with the out of sample accuracy. Using higher resolution images compared to training, they achieved 77% as their max possible accuracy (Dwivedi et al., 2023).

A key paper that aided our analysis is *CarDD: A New Dataset for Vision-Based Car Damage Detection,* by Wang et al. Unlike the previous ones, this did not present any model architecture. However, it presents a dataset: the CarDD Dataset, which suits our use case very well. The dataset it presents is big, comprehensive, and readily labeled, with both bounding boxes and object masks (Wang et al., 2023). This newly available data along with more recent development of object detection model architectures could lead us to a better solution in localizing a vehicle's visual defects.

# Data

## Data Sources

Our core dataset used for our model training will be the CarDD dataset. As mentioned beforehand, this is a new, comprehensive dataset on vehicle visual damage detection. The dataset contains 4,000 labeled images, with each image containing one or multiple vehicle defects. Note that we will not be using videos as our training data since our model is essentially intaking image frames of the video and these labeled images are our best current available data.

Below is an example of the plain image from the dataset without annotations, and the bounding box annotations drawn on according to the coordinates stored in JSON format:

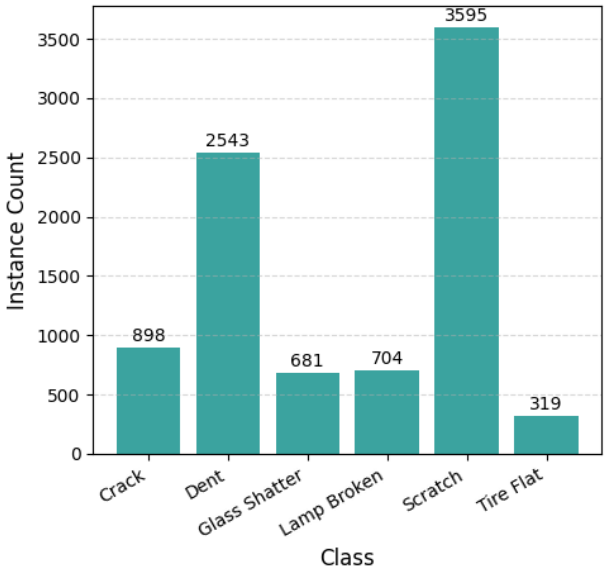
**Figure 2.** *Example of data image and labels*

**Descriptive Analysis**

The Car Damage Detection (CarDD) dataset contains six defect categories: scratch, dent, crack, glass shatter, lamp broken, and tire flat. Each image includes both YOLO-format bounding boxes and object masks, which supports analysis for classification, detection, and segmentation tasks. Our descriptive analysis examined the distribution of defect types, image characteristics, and potential sources of bias that may influence model performance.

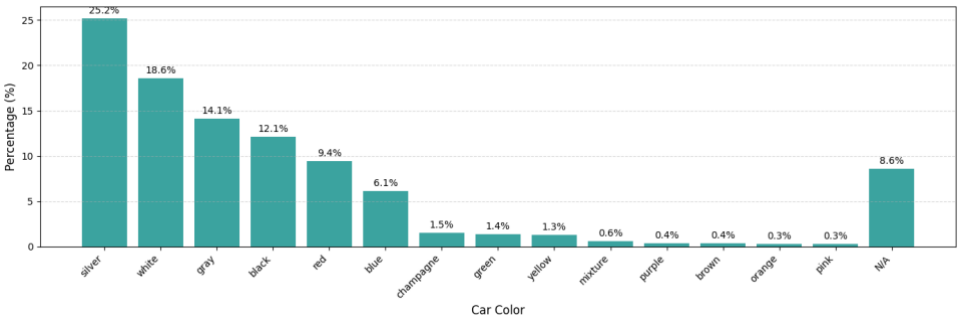
One key finding is the significant class imbalance across defect types. Scratches and dents appear much more frequently than cracks, tire flats, or glass shatter. This imbalance has direct implications for model training because overrepresented classes may dominate the loss function and bias predictions unless balancing or augmentation techniques are used. These observations guided the targeted augmentation strategies applied later in the project.

**Figure 3.** *Class imbalance in damage type*



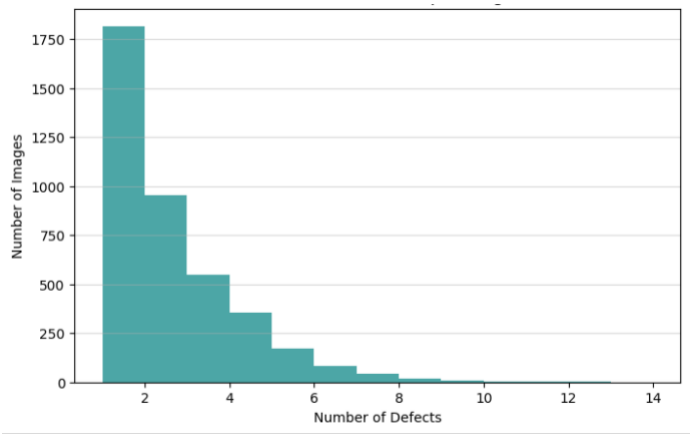

We also evaluated the distribution of vehicle colors because surface color can affect defect visibility. The dataset is dominated by silver, white, and black vehicles, while less common colors appear far less often. Models trained on this distribution may not generalize well to vehicles with uncommon colors. This observation supports the need to vary lighting and color conditions during augmentation to improve model robustness.

**Figure 4.** *Class imbalance in vehicle color*



Additional characteristics of the dataset highlight further modeling challenges. Most images have a resolution of about 1000 × 667 pixels, which provides enough detail for detecting small defects but requires efficient preprocessing for large-scale training. The dataset contains an average of 2.19 defects per image and shows a skewed distribution. Many images contain only one small defect, while others contain several overlapping defects on heavily damaged vehicles. This variation affects bounding box density and label quality, which are important considerations for oriented object detection.

**Figure 5.** *Distribution of defects per image*




Overall, the descriptive analysis identified important patterns including class imbalance, color bias, angle bias, and variability in defect density. These findings directly informed our feature engineering decisions, especially the use of rotation, hue shifting, noise augmentation, and oriented bounding box modeling to better capture the geometry and visual variability of real defects.

# Methodology

## Feature Engineering

Various feature engineering methods were used to improve the performance, robustness, and generalization of our defect-detection model. These included comprehensive data preprocessing as well as data augmentation.

### ***Data Preprocessing***

*Train/Validation restructuring*

The publicly available **CarDD** dataset provides predefined training, validation, and test splits. After an initial exploratory analysis, we merged the original training set of 2,816 images with the 810-image validation set to form a larger and more diverse training pool of 3,616 images. The original 374-image test set was kept untouched and used both as a validation reference during development and as the final benchmark for performance reporting. This restructuring allowed the model to learn from a broader distribution of examples, which was particularly important due to the notable class imbalance and significant variation in defect appearance within the dataset.

*Additional images*

To further improve robustness under real-world factory conditions, we incorporated manually curated supplementary images. We collected 40 undamaged car images from online sources to provide the model with examples of normal vehicle surfaces. These helped

counterbalance the dataset's bias toward heavily damaged vehicles and reduced the frequency of false positives. In addition, we included 20 single-defect images, each containing one clear instance of dent, scratch, or crack, to support a qualitative visual evaluation. These images served as controlled test cases that allowed us to inspect how the model behaves on isolated, high-visibility defects similar to those expected in production environments. Together, these additions improved the model's ability to distinguish between defective and non-defective regions and strengthened detection performance on cleaner backgrounds.

*Label corrections*

During dataset review, we observed inconsistencies and missing annotations, especially in images labeled with scratch or crack defects. These defect types often span large areas, appear thin or low contrast, or include fragmented patterns, which can lead to incomplete or inaccurate bounding boxes. To ensure reliable evaluation, we manually corrected incorrect and missing annotations within the test set. This step was necessary to prevent performance metrics such as mAP from being skewed by label noise, and to ensure a fair and meaningful comparison between model predictions and ground-truth labels.

*Label processing for OBB training*

The original CarDD annotations are provided in COCO JSON format using axis-aligned bounding boxes defined by center coordinates, width, and height. Training YOLOv11-OBB required converting these annotations into formats suitable for oriented bounding box prediction. First, we converted COCO annotations into the standard YOLO format by normalizing bounding

box coordinates to the [0,1] range and generating class-first text labels compatible with the Ultralytics pipeline. Next, we transformed these axis-aligned YOLO annotations into the **DOTA OBB format**, recomputing the center + height + width values into the coordinates of the four corners of the bounding box, being compatible with OBB model's output.

***Data Augmentation***

*Increasing data diversity and reducing class imbalance*

To expand the number of crack, scratch, and dent samples and expose the model to more realistic variation, we implemented several manual augmentations.

First, we applied **rotation**, generating +30° and –30° variants of images containing the three defect types. After rotation, we recomputed the oriented bounding box (OBB) annotations to ensure accurate supervision. This augmentation improved the model's ability to detect angled, slanted, and diagonal defects that commonly occur in real settings.

We also applied **hue shifting**, adjusting color tones specifically for crack images to simulate variations in lighting, reflections, and color balance. This enhanced the model's robustness to shadows, glare, sensor-induced color changes, and different colored cars.

Finally, we introduced **Gaussian noise**, adding light pixel-level noise to mimic minor camera or sensor imperfections. This helped prevent overfitting by ensuring the model did not overly depend on a few high-res specific patterns that are not generalizable outside our dataset.

Together, these augmentations significantly improved performance for small, subtle, and low-contrast defect classes.

**Figure 6.** *Original image before data augmentation*

**Figure 7.** *Image after rotation*

**Figure 8.** *Image after hue shift*

*Reducing overfitting with YOLO's built-in augmentations*

In addition to manual augmentation, we leveraged YOLO's built-in real-time augmentation pipeline. These augmentations are applied probabilistically during training, allowing the model to encounter a unique variation of each image during every epoch, which greatly increases data diversity without requiring additional storage. One of the most impactful techniques is **Mosaic augmentation**, which merges four different images into a single composite. This significantly enriches the training signal, improves the model's ability to detect small defects, and exposes the network to a wide range of spatial arrangements and contexts.

Other augmentations further strengthen the model's robustness. **Horizontal flipping** improves left–right viewpoint invariance, while **translation** shifts the image slightly to prevent

the model from relying on fixed positional cues. **Scaling** applies random zooming in or out to enhance multi-scale detection performance. Additionally, **HSV color jittering** perturbs hue, saturation, and brightness, enabling the model to remain resilient under varying lighting conditions and color distortions typical in real-world inspection environments.

Together, these augmentations substantially reduce overfitting and improve generalization across all defect types.

## Modelling Framework

### *Architectural Overview*

Fundamentally, the You Only Look Once (YOLO) framework represents a paradigm of single-stage object detection, characterized by its ability to predict bounding boxes and class probabilities in a single forward pass, thereby facilitating real-time inference. The architecture partitions the input image into an S grid, where each cell is responsible for predicting objects, whose geometric centers fall within its boundaries. For every grid cell, the network outputs a set of candidate bounding boxes parameterized by their center coordinates (x, y), width w, height h, and a confidence score, alongside class probabilities. This results in an output tensor of dimensions $S * S * (5B + K)$, where B denotes the number of bounding boxes per cell and K represents the number of distinct classes. A subsequent post-processing stage employs confidence thresholding and Non-Maximum Suppression (NMS) to eliminate low-confidence predictions and redundant overlaps, ensuring that only the most reliable detections are retained.

### *The YOLOv11-OBB Variant*

Building upon this foundational architecture, this study adopts YOLOv11, a recent iteration of the YOLO family. This version integrates an anchor-free detection head, a highly efficient backbone, and enhanced feature aggregation mechanisms, which collectively optimize both detection accuracy and inference latency. Critically, we utilize the YOLOv11-OBB (Oriented Bounding Box) variant. Unlike traditional detectors that generate axis-aligned rectangles, the OBB formulation predicts rotated bounding boxes. This capability is essential for automotive defect detection, as it allows the model to accurately capture the orientation and aspect ratio of elongated or slanted defects, such as scratches and cracks, on curved vehicle surfaces. By minimizing background noise and improving Intersection over Union (IoU) calculation for rotated or irregularly shaped damage, the OBB approach offers superior localization compared to standard horizontal bounding boxes.

### *Model Configuration and Initialization*

Model engineering began with model selection. We evaluated multiple sizes of the YOLOv11-OBB architecture, comparing smaller variants to larger ones in order to balance the risks of underfitting and overfitting. Compact models such as YOLOv11s provide an appealing trade-off between accuracy and speed, which is relevant for real-time manufacturing applications. However, our primary experimental focus was on the larger YOLOv11-XL OBB variant, as it offered the strongest empirical performance across defect categories and allowed us to explore the upper limits of accuracy under relaxed computational constraints.

All models were initialized using pretrained weights derived from large-scale datasets such as COCO and relevant defect-detection corpora. This transfer learning strategy enabled the network to begin training with robust visual feature representations, significantly reducing the required training time and data volume needed for convergence on the CarDD dataset. This modeling framework, combining a real-time single-stage detector, oriented bounding box prediction, and pretrained backbones, provides a strong foundation for accurate and efficient visual defect detection in automotive manufacturing settings.

Following model selection, we performed hyperparameter optimization using a targeted grid search. This process identified the optimal training parameters, including an image size of 672, a batch size of 16, and the AdamW optimizer. The optimizer was configured with an initial learning rate of 0.001 and a final learning rate factor of 0.01 using cosine decay, which produced stable and efficient convergence while operating within the GPU memory constraints of our environment.

Training stabilization techniques were applied to further enhance model robustness. We also set a 0.05 label smoothing to regularize our weights, helping to prevent too much overfitting. Mosaic augmentation was used during the early phase of training to expose the model to a wide range of spatial layouts. We then set the close_mosaic parameter to 70 so that the final epochs of training would focus on clean and non-distorted images, allowing the network to refine its predictions on more realistic data.

Together, these engineering decisions produced a stable, high-performing defect detection model that is suitable for deployment in real-world automotive inspection systems.

### *Evaluation Metrics*

To rigorously evaluate and validate our vehicle defect detection model, we used a structured approach combining quantitative metrics and qualitative analysis. Our primary quantitative measure was mean Average Precision (mAP) at an IoU threshold of 0.5, which evaluates how closely predicted bounding boxes match the true defect locations. Higher IoU scores indicate better localization performance, and mAP is widely used in object detection to measure overall prediction accuracy. Our target was to achieve at least 0.65 mAP@0.5 to ensure reliable classification and detection of vehicle defects.

Qualitative evaluation further supported model assessment. Randomly sampled frames were manually reviewed to examine the clarity, reliability, and usefulness of the model's predictions. In addition, systematic error analysis was conducted to identify misclassifications and missed defects, particularly in challenging conditions such as varied lighting or complex angles. These insights guided iterative improvements that strengthened the model's real-world performance.

### *Application Architecture and System Workflow*

The end-to-end prototype developed for this project included a fully functional application designed to simulate how an AI-powered defect detection system would operate within an automotive inspection environment. Although the primary analytical focus of this work

was model development, the accompanying application played a critical role in demonstrating feasibility, user workflow, and the operational value of an automated visual inspection pipeline. The system integrated video ingestion, frame extraction, defect inference, and interactive visual review within a cohesive software architecture built using Django for the backend and HTML/CSS/JavaScript for the frontend.

At a high level, the application workflow began with inspection videos uploaded through a web interface. These videos mimicked those captured on manufacturing lines where an operator or mounted camera moved around the vehicle. Because raw inspection videos often contained large amounts of redundancy—particularly when the camera lingered over the same surface—the backend processed each video into individual frames and applied a Structural Similarity Index (SSIM)–based filtering method to remove frames that did not add meaningful visual information. Consecutive frames were compared using SSIM, and when the similarity exceeded a predefined threshold, the latter frame was discarded. This approach reduced noise, lowered computational cost, and ensured that only structurally informative frames were passed to the detection model. The SSIM module was implemented as a core component of the Django backend and was triggered automatically upon video upload without requiring user intervention.

Once the curated set of frames was produced, the backend ran defect inference on each frame using the trained detection model. For every frame, the system generated bounding box coordinates, orientation information from the OBB predictions, defect types, and confidence scores. These outputs were saved in a structured JSON format and linked to a unique processing

job identifier. The backend exposed this data to the frontend through a set of REST APIs that cleanly separated computational tasks from the user interface. This modular design ensured that improved model versions or alternative detection engines could later be integrated without requiring major changes to the frontend.

The HTML/CSS/JavaScript frontend provided an intuitive visualization environment in which users could view, navigate, and interpret processed video frames. After uploading a video, users were presented with an automatically generated gallery of extracted frames arranged in an interactive grid. Frames containing detected defects were visually highlighted so that inspectors could quickly identify areas requiring attention. Selecting a frame opened a larger preview pane displaying the original image with all associated defect annotations rendered as oriented bounding boxes. The preview pane included padding to preserve aspect ratio and visually resemble the source environment. Users were also able to filter frames by defect type and toggle annotation visibility. The interface emphasized clarity and usability through consistent color styling, smooth transitions, and responsive layout design.

**Figure 9.** *Example of the web interface*

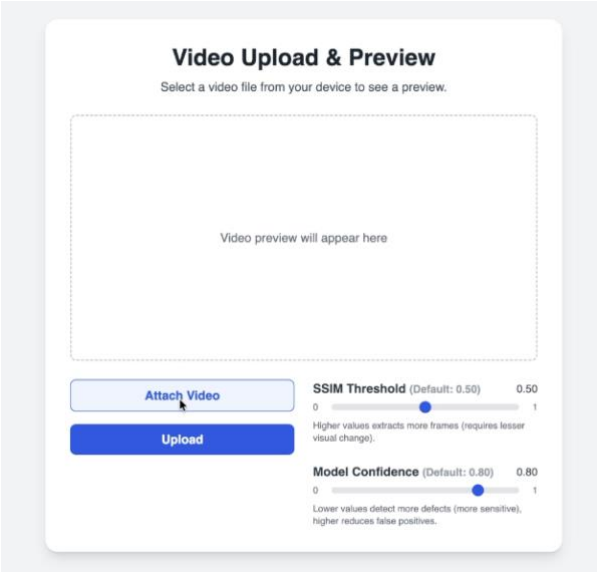
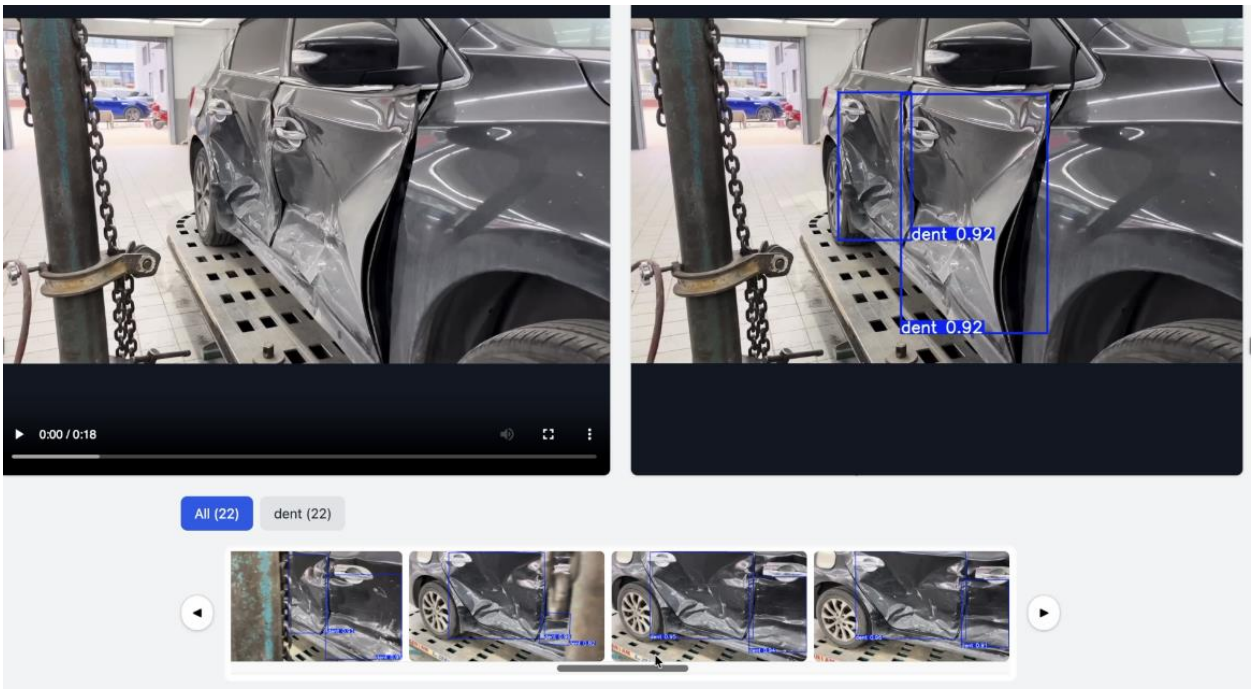



**Figure 10.** *Example of the web interface results*



Although the initial project plan included deploying the system on Google Cloud Platform (GCP), this approach was revised in alignment with the client's internal infrastructure requirements. Since deployment and ongoing maintenance fall within the responsibilities of the client's full-stack engineering team, we concluded the project by delivering the complete frontend and backend codebases, along with documentation, to their designated repositories. The delivered application remained fully operational in a local development environment, supporting video upload, automated frame extraction, model inference, and interactive defect visualization. This ensured that the client's engineering team could proceed with deployment and integration without loss of functionality.

***Frame Extraction Using Structural Similarity Index (SSIM)***

A key component of the system pipeline was the automated extraction of representative frames from inspection videos. Real-world inspection footage typically contained a high degree of redundancy because the camera moved slowly around the vehicle, often covering the same region for several consecutive seconds. Processing every frame not only increased computational cost but also introduced noise that could degrade detection performance. To address this challenge, the system incorporated the Structural Similarity Index (SSIM) as the basis for filtering redundant frames and retaining only those with meaningful structural differences.

SSIM is a perceptual similarity metric that evaluates images based on luminance, contrast, and structural similarity. Structural information is particularly important for detecting scratches, dents, and other surface anomalies. SSIM outputs a score between −1 and 1, with

values closer to 1 indicating nearly identical images. In inspection videos, adjacent frames often achieved SSIM scores above 0.90 due to minimal changes in viewpoint or lighting.

In the implemented workflow, each new frame was compared against the most recently accepted frame. If the SSIM score exceeded a predefined threshold, the new frame was classified as redundant and discarded. Only frames showing meaningful structural differences were retained. This threshold was selected empirically to balance the preservation of diverse viewpoints with the need to eliminate near-duplicate frames. By applying SSIM in this manner, the system significantly reduced the number of frames sent to the detection model, improving computational efficiency without sacrificing coverage of the vehicle's surfaces.

This SSIM-driven approach proved essential for reproducing realistic inspection conditions. It ensured that the system focused on structurally informative perspectives that human inspectors naturally consider important while automatically removing repetitive content. The resulting curated frame set provided a cleaner and more efficient input for the defect detection model, ultimately enhancing overall system performance.

## Findings

We deployed the YOLO11-XL (OBB) architecture and trained it on the CarDD dataset for 140 epochs. The training configuration utilized an input image size of 672 x 672 pixels, a batch size of 16, a confidence threshold of 0.1, and an Intersection over Union (IoU) threshold of 0.5. The model's performance was evaluated using standard object detection metrics, achieving

an overall mAP@0.5 of 0.824 and an mAP@0.5:0.95 of 0.707. The aggregate precision and recall were 0.858 and 0.759, respectively.

**Table 1.** *Evaluation Results*

| Class | Images | Instance | Precision | Recall | mAP@0.5 | mAP@0.5:0.95 |
|---|---|---|---|---|---|---|
| **All Classes** | 374 | 804 | 0.858 | 0.759 | 0.824 | 0.707 |
| **Dent** | 158 | 241 | 0.821 | 0.680 | 0.789 | 0.548 |
| **Scratch** | 186 | 321 | 0.735 | 0.642 | 0.708 | 0.481 |
| **Crack** | 49 | 70 | 0.719 | 0.586 | 0.685 | 0.525 |
| **Glass Shatter** | 71 | 71 | 0.986 | 0.968 | 0.984 | 0.963 |
| **Lamp Broken** | 65 | 69 | 0.888 | 0.841 | 0.860 | 0.821 |
| **Tire Flat** | 31 | 32 | 1.000 | 0.840 | 0.919 | 0.907 |

The experimental results demonstrated that the YOLO11-XL (OBB) model achieved high accuracy in detecting visually distinct structural damages. Classes such as glass shatter and tire flat exhibited particularly strong performance, with mAP@0.5 scores of 0.984 and 0.919, respectively.

In contrast, the model found smaller and more localized defects—such as dents, scratches, and cracks—more challenging to detect. This was reflected in comparatively lower mAP@0.5 scores, such as 0.708 for the scratch class. These findings align with the inherent difficulty of identifying thin, low-contrast, and irregular defect patterns.

## Discussion

### Impact of label ambiguity on mAP@0.5 and mAP@0.5:0.95.

In this study, we reported both COCO-style mAP@0.5:0.95 and the simpler mAP@0.5, but we primarily interpreted model quality through mAP@0.5. This choice was motivated by the fact that, despite our manual efforts to correct clearly erroneous annotations in CarDD and in the additional images we labeled, there is still no fully standardized annotation protocol for fine-grained defects such as cracks and scratches. Different annotators sometimes draw a single large box for an extended scratch, while others split the same physical defect into several smaller boxes; in other cases, the ground-truth box may be slightly shifted or loosely aligned around a very thin crack. Under these conditions, a prediction that a human inspector would consider correct can easily fall below stricter IoU thresholds and be counted as a false positive or false negative.

**Figure 11**. *Ground Truth Labels VS Model Prediction*

Figure 11 illustrates this effect: the model prediction captures both the scratch region and the crack near the lamp in a way that would be acceptable for an operator on the line, yet several of these detections do not match any ground-truth box with IoU ≥ 0.5 because the annotation granularity and box placement differ. As a result, mAP@0.5:0.95 becomes a conservative lower bound that is highly sensitive to small localization discrepancies and annotation noise, especially for thin, elongated defects. In contrast, mAP@0.5 is more robust to these inconsistencies and better reflects the practical capability of the detector in our current dataset setting. Consequently, we used mAP@0.5 as the main metric for interpreting model performance, while still reporting mAP@0.5:0.95 for comparability with standard benchmarks.

**Evaluation of Alternative Architectures**

To ascertain whether an end-to-end, transformer-based architecture could yield superior performance relative to our convolutional OBB baseline, we replicated the complete training procedure using RT-DETR. Specifically, the Ultralytics implementation of RT-DETR was trained for 100 epochs on the augmented CarDD COCO-style dataset (data_aug2.yaml).

To ensure a direct and valid comparison, all experimental parameters were held constant with the YOLOv11-OBB runs, including imgsz=640, batch=16, and the A100 compute environment. This configuration, representing our optimal RT-DETR setting, produced the following test split metrics: P=0.839, R=0.710, mAP@0.5=0.777, and mAP@0.5:0.95=0.619.

Although these results are robust, particularly for high-visual-saliency classes ('glass shatter', 'tire flat', 'lamp broken'), the primary mAP@0.5 metric (0.777) remained quantitatively inferior to our YOLOv11-OBB benchmarks. On identical data, the YOLO11-XL (OBB) achieved mAP@0.5:0.95=0.707 (with P≈0.86, R≈0.76), and the mAP@0.50 improved to 0.824.

Our analysis revealed that RT-DETR's performance deficit was most pronounced in the more challenging, long-tail defect categories ('dent', 'scratch', 'crack'). Given that these specific classes are the principal drivers for our business case, we retained the YOLOv11-OBB (XL variant) as the primary model. The RT-DETR architecture was consequently classified as a verified but non-dominant alternative in our model repository.

**Application Integration**

The application developed as part of this project demonstrates not only the feasibility of automated defect detection but also the practicality of integrating such technology into an inspection workflow. By combining SSIM-based frame extraction with a structured interface for visual review, the system closely mirrors the decision-making patterns of human inspectors while reducing the manual burden associated with frame-by-frame evaluation. The separation between the computational backend and the visualization frontend proved advantageous,

enabling each component to be iterated on independently and positioning the system for smooth transition into a cloud or enterprise environment. Feedback collected during internal demonstrations indicated that the interface's clear annotation overlays, consistent styling, and intuitive filtering tools significantly improved users' ability to navigate inspection outputs compared to traditional manual inspections. The prototype thus validates not only the underlying model's performance but also the end-user experience required for successful adoption of AI-assisted quality control. By delivering the complete application codebases to the client's repositories, we ensured a seamless handoff to their full-stack engineering team, who will handle deployment and long-term operationalization of the system within their internal infrastructure.

## Conclusion

This study shows that the YOLO11-XL OBB model is highly effective for automated vehicle defect detection on the CarDD dataset. With data preprocessing, targeted augmentation, and careful model engineering, the system achieved strong performance across all defect types, with every class exceeding 0.65 mAP@0.5. The model performs especially well on clear and visually distinct defects such as glass shatter, lamp damage, and tire flat issues, demonstrating strong precision and overall accuracy.

More subtle defects, including cracks, scratches, and dents, remain more challenging due to their small size, thin structure, and inconsistent labeling in the dataset. Even so, targeted augmentations and improved training methods boosted performance for these classes.

Overall, the results indicate that YOLO11-XL with oriented bounding boxes provides a solid foundation for real-world automotive defect detection. With cleaner, domain-specific data, the system can be further strengthened and adapted to production environments. Combined with the full end-to-end pipeline developed in this project—from data processing and training to deployment and visualization—the solution can significantly improve the efficiency, speed, and consistency of vehicle inspection workflows.

This positions LatentView Analytics to offer a commercially valuable AI-based inspection product that can be adopted by automotive manufacturers, insurance companies, used-car evaluators, and fleet operators, bringing measurable operational benefits across the industry.

**Future Work**

The current OBB-based YOLOv11 prototype already delivers reliable performance on visually salient defects and could support an initial deployment in a pilot production setting. To further strengthen robustness on inconspicuous damage types, such as fine cracks and light scratches, future work will focus on two directions: upgrading the model architecture and enhancing the representativeness of the training data.

**1. Architectural Evolution**

A first direction for future work is to upgrade the backbone architecture while maintaining a consistent training and evaluation pipeline. In particular, the upcoming YOLOv12-OBB framework can be integrated and benchmarked against the optimized YOLOv11-OBB

baseline. By retaining the most effective preprocessing techniques, augmentation strategies, and hyperparameter settings from the current system, any observed performance gains can be attributed directly to architectural advancements rather than variations in training configurations.

In addition, next-generation architectures can be evaluated under a controlled protocol. Beyond the YOLO family, this includes improved DETR variants and other state-of-the-art detectors. To ensure an equitable comparison of mean Average Precision (mAP), latency, and resource utilization, all candidate models would be trained using identical input resolutions and confidence thresholds. Such experiments will clarify whether newer architectures can resolve detection issues for small, localized defects while still satisfying the strict real-time constraints of the inspection line.

**2. Data Refinement and Domain Adaptation**

A second priority is to align the training data more closely with actual assembly-line conditions, particularly for rare classes where the current model underperforms.

***Real-World Client Data and Targeted Defect Coverage***

Within the current project, training and evaluation were based on the CarDD dataset, which was further enriched with a small number of additional images and annotations created by our team following the same defect schema. On top of this, class-balancing data augmentation was applied by generating multiple variants of images containing dent, scratch, and crack defects, partially compensating for their low base frequency. These steps push the benchmark

data closer to realistic inspection scenarios, but they still remain surrogates for truly representative observations from the production line. In a deployed setting, a natural next step is to construct a targeted dataset from real client manufacturing data: inspectors would deliberately capture and label micro-scratches, hairline cracks, and minor dents on frequently inspected body panels under normal line conditions, and the detector would be fine-tuned on these images and labels. Such a client-specific dataset, built from real inspection imagery rather than augmented surrogates, would complement the extended CarDD corpus and help close the remaining domain gap between benchmark experiments and real-world operation.

***Label Quality Enhancement.***

While this proof-of-concept study utilized publicly provided CarDD annotations and supplementary labels as the primary supervision signal, residual label noise and bounding box inaccuracies persist despite manual corrections. These imperfections likely constrain performance, particularly regarding small and ambiguous defect instances. Therefore, future research initiatives should prioritize a more systematic quality assurance pipeline to enhance label fidelity. Recommended strategies include the establishment of rigorous annotation guidelines for each defect category and the implementation of targeted reviews for high-loss or high-uncertainty examples. Furthermore, the construction of a high-precision gold standard subset through dual-annotation and adjudication is advisable. The adoption of such a structured label quality framework is anticipated to yield significant improvements in both model calibration and recall for challenging defect categories.

## 3. Future Application Development

Future development of the application will be carried out by the client's full-stack deployment team, who will integrate the delivered prototype into their existing cloud infrastructure. Their next steps will involve adapting the backend for scalable GPU-enabled inference, configuring authentication and access control, and optimizing the system for continuous inspection workflows. From a broader perspective, further enhancement of the platform may include support for live video streams, long-term inspection data storage, and automated defect reporting. These additions will allow the system to evolve from a functional prototype into a turnkey inspection tool suitable for enterprise-level manufacturing environments.

## References


1. Averroes AI. (n.d.). *Automated quality control vs manual inspection*. Retrieved May 2, 2025, from https://averroes.ai/blog/automated-quality-control-vs-manual-inspection
2. Dwivedi, M., Hashmat, M., Shadab 2, Omkar, S. N., & Rathi, A. (2019, September 23). *Deep Learning Based Car Damage Classification and Detection*. https://doi.org/10.13140/RG.2.2.18702.51525
3. Intellinet Systems. (n.d.). *Traditional inspection methods vs digital inspection software*. Retrieved May 2, 2025, from https://www.intellinetsystem.com/blogs/traditional-inspection-methods-vs-digital-inspection-software
4. Jegham, N., Koh, C. Y., Abdelatti, M., & Hendawi, A. (2024). *Evaluating the Evolution of YOLO (You Only Look Once) Models: A Comprehensive Benchmark Study of YOLO11 and Its Predecessors*. arXiv preprint arXiv:2411.00201. Retrieved from https://arxiv.org/abs/2411.00201
5. LatentView Analytics. (n.d.). *ConnectedView*. Retrieved May 2, 2025, from https://www.latentview.com/connectedview/
6. LatentView Analytics. (n.d.). *Data Analytics consulting company in the US | AI and Data Analytics Solutions by LatentView*. Retrieved May 2, 2025, from https://www.latentview.com/